



Universidad Nacional Autónoma de México

Facultad de Ciencias

Práctica Número 6

Cristian Guillermo Mejia Cisneros

Número de cuenta: 427329650

30 de octubre del 2026

1. Objetivos

- **Entrada de datos**: Usar `Scanner` para leer datos de distintos tipos que el usuario escribe en la terminal.
- **Validación de datos**: Revisar que los datos que ingresa el usuario sean del tipo esperado y tengan sentido, y volver a pedirlos cuando no lo sean, sin que el programa truene.
- **Integración**: Combinar variables, condicionales, ciclos, `printf`, la biblioteca `Math` y la entrada de datos en un programa completo e interactivo.

2. Desarrollo

Parte 1: Conociendo Scanner

Actividad 1.1 - Dardos a la medida

Vamos a comenzar la práctica investigando un poco los métodos `nextInt()` y `hasNextInt()`. La manera de explicar de manera breve ambos, será respondiendo las siguientes preguntas:

1. ¿Qué hace cada uno de estos métodos y qué valor regresa?

   Respuesta: Según la Documentación de Scanner, `nextInt()` es un método dentro de la clase `Scanner`, analiza que el siguiente valor en la entrada sea un entero, es decir, dependiendo de como lo especifiquemos, si nosotros le damos a revisar un valor (ya sea , indicándole desde donde tiene que leer los datos, como puede serlo desde la misma entrada de la consola), y encuentra que este es un entero, regresará este mismo valor entero.

   Para el caso de `hasNextInt()`, de igual manera es un método dentro de la clase `Scanner`, que hace exactamente lo mismo que `nextInt()`, solo que, si encuentra que el número analizado es un entero, regreso el valor de verdad `true` y en caso contrario el valor de verdad `false`.

2. ¿Qué pasa si se una `nextInt()` y el usuario escribe algo que no es un número entero?

   Respuesta: Para responder la siguiente pregunta, hemos realizado un programa con el siguiente código:

```java
import java.util.Scanner;
public class pruebaEntero {

    public static void main (String [] args){

        Scanner introducirEntero = new Scanner (System.in);
        int entero;

        System.out.println("Introduce un entero: ");

            entero = introducirEntero.nextInt();

        System.out.println("Numero: " + entero);
    }
}
```

El programa anterior, lee un número entero ingresado por el usuario y lo imprime. Notemos que, para ocupar `Scanner` y todos sus métodos, hay que importarlos mediante `import java.util.Scanner`, no es como con `Math` que ya estaba cargado por defecto. Además, para ocupar `Scanner`, dado que es un clase, debemos instanciarla, con un objeto al que nombramos `introducirEntero`, que indica que será un nuevo `Scanner` utilizado para poder ocupar los distintos métodos de dicha clase, en pocas palabras, `introducirEntero` servirá para mandar a "llamar", las distintas herramientas que necesitemos. El "`System.in` ", no es más que un método de la clase `System` de Java que nos ayudará a indicar que se reciben datos a través de alguna entrada. La clase `System` al igual que `Math`, no necesitamos importarla, más aun, según la documentación oficial de System, no es necesario instanciarla.

Se que parece alargada la respuesta a la pregunta 2, pero queremos dejar en claro que se esta haciendo, para poder usarlo sin volverlo a explicar, en los siguientes problemas a resolver. Finalmente, para usar `nextInt()` de tal manera que escanee lo que el usuario ingresa en la terminal, mandamos a llamar a `nextInt()`, mediante nuestro objeto creado, con la siguiente sintaxis: `introducirEntero.nextInt();`, colocado lo anterior, en la ejecución, la terminal esperara recibir un dato en ese momento.

Regresando a la pregunta principal, con este programa puesto a prueba, si ingresamos un valor como "a", recibiremos el siguiente error:

```
    Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
        at java.base/java.util.Scanner.next(Scanner.java:1602)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
        at pruebaEntero.main(pruebaEntero.java:13)
```

Pues el dato ingresado no es un entero.

3. ¿Cómo puede ayudarte `hasNextInt()` a evitar este problema?

Respuesta: El método puede ayudarnos precisamente, a verificar si lo que ingreso el usuario es un número entero, o si por el contrario, a escrito un valor no válido, veamos el siguiente código:

```
import java.util.Scanner;
public class pruebaEntero {
    public static void main (String [] args){
        Scanner introducirEntero = new Scanner (System.in);
        int entero;
        System.out.println("Introduce un entero: ");
        if (introducirEntero.hasNextInt() == true){
            entero = introducirEntero.nextInt();
}
else {
        System.out.println("El dato ingresado no es valido");
}
    }
}
```

El código anterior, es una modificación al presentado en la pregunta **2**, solo que ahora, antes de ingresar un valor `entero`, primero que nada, verifica si es valido con `hasNextInt()`,

el anterior método, no guarda ningún valor, solo verifica si es entero, dejando la información en espera, si resulta `true`, `nextInt()`, ahora si tomará el resultado y lo guardara en entero.

Explicación del código

Para realizar el programa, prácticamente el único cambio relevante, esta en la parte en la que el usuario ingresa la cantidad de dardos a ingresar, pues para la parte de la estimación de π, sigue siendo el mismo código de la `Practica05`, que es el siguiente:

```
        while(dardosLanzados < dardosALanzar){
        abscisa = Math.random();
        ordenada = Math.random();
        distanciaAlOrigen = Math.pow(abscisa,2) + Math.pow(ordenada,2);

            if (distanciaAlOrigen <= 1.0){
                dardosDentro += 1;
                }
        dardosLanzados ++;
            }

estimacionPi = 4.0 * dardosDentro/dardosLanzados;


System.out.println("\nDardos lanzados: " + dardosLanzados);
System.out.println ("\nDardos que cayeron dentro del circulo: "+
   dardosDentro);
System.out.printf("\nEstimacion de Pi: %.6f \n", estimacionPi);
```

No nos detendremos a explicar el anterior bloque de código, pues este fue expuesto en la práctica, pasada, los cambios relevantes fueron los siguientes:

```
    while (introducirEntero.hasNextInt() != true ){
        System.out.println("\nEl valor '" + introducirEntero.next()  +
           "' No es valido, es un caracter o double ingrese otro: \n");
    }

dardosALanzar = introducirEntero.nextInt();

    while(dardosALanzar < 0){
        System.out.println("\nEl valor '" + dardosALanzar + "' No es
           valido, es un entero negativo, ingrese otro: \n");
        dardosALanzar = introducirEntero.nextInt();
    }
```

Utilizamos dos `while`, para comprobar que los datos ingresados no sean incorrectos. En el primer `while` del bloque código, comprobamos si el usuario no esta ingresando nada que no sea un número entero, mediante `introducirEntero.hasNextInt()`. Mientras el usuario no ingrese nada que no sea un dato correcto, este seguirá pidiendo otro valor al usuario.

Cuando sale por fin de este ciclo, con ayuda de `dardosALanzar = introducirEntero.nextInt();`, guardamos el ahora número entero correcto que el usuario ingreso, la instrucción anterior podría parecer a simple vista que le esta volviendo a pedir al usuario el dato, o por lo menos asi lo pensamos, pero `hasNextInt()` solo verifica si es entero o no, es decir, únicamente regresa un

valor de verdad, no guarda el valor del entero, este queda en el "vacio" o en el bufer y finalmente, al utilizar `introducirEntero.nextInt()` podemos guardar el valor en la variable que hemos declarado previamente.

El siguiente `while` del bloque comprobará si nuestro dato es un entero positivo. Dado que los enteros, pueden ser negativos o positivos, `hasNextInt()`, dará como válido un valor negativo, pues a pesar de su signo, sigue siendo un entero, por lo tanto este nuevo `while` se mantendrá en bucle hasta que el usuario ingrese un valor válido para este punto, pues lógicamente, no podemos arrojar una cantidad negativa de dardos.

En **PiInteractivo.png**, dentro de nuestro directorio `Practica06/images`, podremos ver las capturas tomadas de la ejecución de nuestro programa, al colocar en la terminal un dato válido y uno no válido con sus respectivos resultados.

Actividad 1.2 - Decimal a binario interactivo

Para la siguiente parte de la prráctica, realizamos el siguiente código:

```
1     import java.util.Scanner;
2
3  public class BinarioInteractivo{
4     public static void main (String [] args){
5
6  int numeroIngresado;
7  char opcionPrograma = 'S';
8  String numeroBinario;
9  Scanner introducirEntero = new Scanner (System.in);
10
11 do{
12     System.out.println("\n========DE ENTERO A BINARIO========\n");
13     System.out.println("Ingrese el numero entero a convertir: \n");
14
15 while (introducirEntero.hasNextInt() != true ){
16     System.out.println("\nEl valor '" + introducirEntero.next()  + "'
          No es valido, es un caracter o no es un numero entero ingrese
          otro: \n");
17    }
18
19 numeroIngresado = introducirEntero.nextInt();
20
21     while(numeroIngresado < 0){
22         System.out.println("\nEl valor '" + numeroIngresado + "' No es
              valido, es un entero negativo, ingrese otro: \n");
23         numeroIngresado = introducirEntero.nextInt();
24     }
25
26 numeroBinario = "";
27
28 while (numeroIngresado/2  != 0){
29
30         numeroBinario = (numeroIngresado%2) + numeroBinario;
31         numeroIngresado = numeroIngresado/2;
32
```

```
33 }
34         numeroBinario = (numeroIngresado%2) + numeroBinario;
35
36 System.out.println("EL NUMERO FINAL ES: " + numeroBinario);
37 System.out.println("\n=====DESEA CONVERTIR OTRO ENTERO A DECIMAL? =====
      ");
38 System.out.println("\n\n-Presione S y 'Enter' para calcular otro entero
      ");
39 System.out.println("\n\n-Presione cualquier otra letra y 'Enter' para
      terminar el programa\n");
40
41 opcionPrograma = introducirEntero.next().charAt(0);
42
43 }while(opcionPrograma == 'S' || opcionPrograma== 's');
44
45 System.out.println("========GRACIAS POR USAR EL PROGRAMA :3========");
46 }
47 }
```

Para explicar a detalle que realizamos en cada caso, vamos a referirnos al número de línea de código, del respectivo bloque de código, anterior expuesto.

Primero que nada, en la línea de código `1`, podemos notar que hemos importado las librerías necesarias para poder utilizar `Scanner`. De las líneas `3-10` no es más que la sintaxis común que utilizamos para comenzar las `clases BinarioInteractivo` y `Main`, para posteriormente declarar, identificar e inicializar las respectivas variables a utilizar.

Saltadonos momentáneamente la línea `11`, de las líneas `15` a `25`, el código es prácticamente igual a lo realizado en la **Actividad 1.1**, solo que adaptado para este caso. Según lo solicitado en la práctica, solo debemos de aclarar al usuario que el valor que ingrese no debe ser, un carácter, un valor decimal o un entero negativo, aclaramos lo anterior pues pese a que en teoría podemos calcular el binario de un número negativo o de un decimal, esto no es lo requerido y por tanto, no hemos propuesto nada más allá de este problema a resolver.

En la línea `26`, estamos asignando a `numeroBinario`, un espacio vació, esto con la finalidad de que si el usuario repite el programa, la variable no contenga el valor del binario calculado anteriormente. El `while` de las líneas `28-33`, es casi todo el proceso que el programa necesita realizar para convertir cualquier entero positivo a un número binario, primero comprobamos que `numeroIngresado/2` sea distinto de cero, esto ya que cuando realizamos el proceso de conversión de entero a binario, de manera escrita, el proceso termina cuando el cociente es igual a 0, pues seguir realizando la división no aportaría ningún número relevante, por ejemplo, si convertimos 4 a binario:

$$(4)_{10} = 2(2) + 0 \tag{1}$$

$$(2)_{10} = 1(2) + 0$$

$$(1)_{10} = 0(2) + 1$$

$$(4)_{10} = (100)_2$$

Sin extendernos mucho en los detalles de como se acomodan los residuos para obtener el número correspondiente binario, notemos que el **2** dentro del paréntesis, solo esta indicando la división entre este mismo número, por lo que, el número a la izquierda de **(2)** no es más que el cociente, y el proceso para cuando este es **0**.

Explicado brevemente lo anterior, y regresando nuevamente a la explicación del último bloque de código colocado, dentro del `while`, vamos almacenando, con ayuda de la variable `numeroBinario`, los residuos de cada división realizada, esto podemos hacerlo, ya que el simbolo `+`, en el caso de `numeroBinario = (numeroIngresado%2) + numeroBinario;`, sirve para concatenar texto, debido al hecho de que la variable de tipo `String`, es más grande que la de tipo `int`, por lo que se hace un **casting** implícito. La razón de no escribir `numeroBinario += (numeroIngresado%2);`, es porque los residuos, queremos guardarlos de derecha a izquierda y no al contrario, para finalmente obtener el binario esperado.

Finalmente **numeroIngresado** toma el valor de el cociente de dividir este número entre 2, y vuelve a ejecutar la evaluación para continuar (o no) con el ciclo `while`. Justo después de la ejecución del `while`, tenemos un `numeroBinario = (numeroIngresado%2) + numeroBinario;`, en la línea `34`, ya que, cuando el cociente es **0**, el ciclo se detiene pero aun queda agregar el residuo de esa división (es decir dividir a 0 entre 2) al binario final, esto es más sencillo de ver en el ejemplo de la **ecuación** (1).

de las líneas `36-40`, solo es la impresión de resultados, lo último relevante del código de este programa son las líneas `43-47`, utilizando `introducirEntero.next.chartAt(0)`, estamos solicitando que se ingrese en la terminal algún carácter, razón por la cual guardamos este valor en `opcionPrograma` que es de tipo `char`. Posteriormente, el `while` de la línea `43` simplemente verifica si ingresamos la opción correspondiente a que el programa se vuelva a ejecutar (en este caso, ingresando la letra "S"), la sintaxis para esta prueba se escribió: `opcionPrograma == 'S' || opcionPrograma== 's'`, para que independientemente de si el usuario escribe s o S, los dos datos los tome como válidos. El `while` anterior no es más que la consecuencia del `do` de la línea `11`, que primero ejecuta todo el programa una ves y después verifica si se necesita ejecutar nuevamente, esto lo realizamos así con el fin de no tener que usar una variable centinela.

Para verificar los resultados de nuestro programa, ingresamos los siguientes datos:

- **Ingresando un entero válido y salir del programa**: Al ingresar el número entero 100 efectivamente realiza la conversión de manera correcta, y si colocamos cualquier otra tecla que no sea **S** y damos enter, el programa termina, todo esto podemos verlo en **Binario1.png**, en la carpeta mencionada con anterioridad.
- **Ingresando datos no validos**: Al ingresar el carácter "t", el número "-5" y "10.5", obtendremos error y no convertirá nada hasta que el usuario ingrese un dato válido, posteriormente si colocamos 100, el programa se ejecutará con normalidad dando el resultado esperado.
- **Repitiendo el programa**: Cuando ingresamos 100, 50 y 0, obtenemos resultados correctos por cada ejecución, el programa se repite de manera esperada, cuando ingresamos "s" o "S", y el valor 0 regresa el número binario 0.

Recordando nuevamente que todas las imágenes correspondientes a las ejecuciones realizadas están en la carpeta `images` que mencionamos en la **Actividad 1.1**.

Parte 2: Bienvenidos a la Cabaña del Misterio

Actividad 2.1 - Programa final para la cabaña del misterio

Para esta sección, debido a que el código es muy extenso, vamos a explicar por bloques de código, descartando los más obvios o no detallando tanto su explicación, cabe aclarar, que al referirnos a pequeñas partes del código fuente, debe ser claro que, algunas llaves, en que clase nos ubicamos o que variables declaramos, no aparecerán, pero todo lo escrito estará dentro del archivo `CabanaDelMisterio.java`, ubicado en el directorio `Practica06/src/`.

. Todas las variables y objetos utilizados son las siguientes:

```
    int mesesPagoCliente1=0;
int contadorErrores=0;
int opcionMenu=0;
int operacionEfectiva=0;

double precioCliente1=0.0;
double interesSimple=0.0;
double interesCompuesto=0.0;
double tasaDeInteres=0.0;
double tasaAnual=0.0;
double totalCompuesto=0.0;
double totalSimple=0.0;
double saldoPorMes=0.0;

char estadoCumplimientoCliente1 = 'N';

boolean numeroValido = true;

String cliente1 = "";

Scanner datosCliente = new Scanner (System.in);
```

Serán detallados conforme aparezcan en la explicación del código. Vamos a comenzar con la siguiente línea de código:

```
nombreCliente = datosCliente.nextLine();
```

Seguido de un `System.out.println(Ïngrese el nombre del cliente: ")`, la anterior línea utiliza `nextLine()`, para leer valores de cadena de texto y guardarla en una variable `nombreCliente`, precisamente `String`, es parecido a lo que hacemos con `nextInt()`, solo que para el caso de texto. Para continuar, tenemos el siguiente bloque:

```
System.out.println("\nIngrese el tipo de cliente: ");
estadoCliente = datosCliente.next().charAt(0);


    while( estadoCliente != 'E' &&  estadoCliente != 'P' &&
       estadoCliente != 'R'){
        if (++contadorErrores >= 3){ break;}
            System.out.println("\n '"+ estadoCliente + "' No es un
               estado de cliente valido, ingrese otro:");
            estadoCliente = datosCliente.next().charAt(0);
    }
```

Al igual que la `Actividad 1.2`, utilizamos `datosCliente.next().charAt(0);` para guardar un carácter, que nos indicará el estatus de cumplimiento del cliente, visto y detallado en practicas anteriores. Cuando el usuario ingresa su dato, este queda guardado en la variable `estadoCliente`, y mediante el `while`, verifica si efectivamente es uno de los valores que el señor Pines había solicitado.

Con ayuda de `&&`, y `!=`, pregunta si el valor ingresado es distinto a cualquier dato válido, si es el caso, entonces debe haber un error de lo que estamos colocando, por lo que `if (++contadorErrores >= 3) { break; }` además de evaluar si ya tenemos 3 errores (es decir, los necesarios para detener el programa), está aumentando `contadorErrores` antes de hacer la comparación, pues ya hemos cometido un error (si es el caso), por lo tanto la comparación es a partir del error que llevamos. A continuación indicamos al usuario que los datos que intenta ingresar no son correctos, pedimos otro dato mediante `estadoCliente = datosCliente.next().charAt(0);`, que al mismo tiempo nos ayuda a actualizar la variable que controla la ejecución del `while`, el dato lo seguirá pidiendo hasta que se ingrese en la terminal un dato válido o en su defecto, se terminen los intentos.

Cuando se termine este ciclo, le sigue el bloque de código:

```
1     if (contadorErrores < 3){System.out.println("\nIngrese el prestamo
          a realizar: ");
2         numeroValido = datosCliente.hasNextDouble();
3     }
4
5     while (numeroValido != true ){
6         if (++contadorErrores >= 3){ break;}
7         System.out.println("\nEl valor '" + datosCliente.next()  + " Es
              un caracter, dato no valido ingrese otro: \n");
8         numeroValido = datosCliente.hasNextDouble();
9     }
```

Primero, nos aseguramos de que los errores no sean los necesarios para terminar el programa, pues si en el ciclo que le precede obtuvimos una cantidad suficiente de errores, no debería de ejecutar ya nada más, ni pedir ningún dato extra o imprimir algo acerca de ingresar uno, ahí es donde entra el `if` colocado (más adelante, para cada dato pedido en este estilo, también colocaremos este `if`, cumple la misma función, por tanto, no se describira con tanto detalle). Con ayuda de la variable `numeroValido` del tipo `boolean` y de `datosCliente.hasNextDouble();` (que realiza lo mismo que `hasNextInt()` solo que para un número decimal), verifica si el dato ingresado es un número, pues si se ingresa un entero o algún negativo será valido, pero si el usuario termina por ingresar un dato erróneo, como lo puede ser un carácter, regresará `false`, hecho por el cual es conveniente guardar este valor en una variable del tipo boolean.

El `while`, al igual que en el caso de actividades pasadas sumado a lo realizado para `estadoCliente`, cumple solo con verificar que el dato ingresado, sea válido. Si resulta que los valores son correctos, tendremos el siguiente bloque de código:

```
1     if (contadorErrores < 3){capitalPrestado = datosCliente.nextDouble
          ();
2     }
3
4     while(capitalPrestado < 0 ){
5         if (++contadorErrores >= 3){ break;}
```

```
        System.out.println("\nEl valor '" + capitalPrestado  + " Es un
            numero negativo, dato no valido ingrese otro: \n");
        capitalPrestado = datosCliente.nextDouble();
    }
```

El `if` nuevamente nos ayuda a verificar, que debemos pedir al usuario datos, siempre y cuando no se haya sobrepasado el límite de errores permitidos. Si aun no llegamos al límite, simplemente, guardamos nuestro número en `capitalPrestado` y con ayuda del `while` que le sigue, verifica que el número ingresado en terminal no sea negativo, siguiendo la misma estructura de los `while`, escritos con anterioridad.

Básicamente todo el código que colocaremos a continuación, sigue la misma estructura, de los bloques que acabamos a explicar, por lo que no nos detendremos a detallarlo, pero si lo escribimos en el presente reporte, con el fin de mostrar, donde se usaron muchas de las variables y que cada dato ingresado fue verificado para que sea correcto.

```
    if (contadorErrores < 3){System.out.println("\nIngrese los meses de
        interes: ");
        numeroValido = datosCliente.hasNextInt();
    }

    while (numeroValido != true ){
        if (++contadorErrores >= 3){ break;}
        System.out.println("\nEl valor '" + datosCliente.next()  + "'
            Es un caracter o un decimal, dato no valido ingrese otro: \n
            ");
        numeroValido = datosCliente.hasNextInt();
    }

    if (contadorErrores < 3 ){mesesPagoCliente = datosCliente.nextInt()
        ;
    }

    while(mesesPagoCliente < 0 && contadorErrores < 3){
        if (++contadorErrores >= 3){ break;}
        System.out.println("\nEl valor '" + mesesPagoCliente  + "' Es
            negativo, ingrese otro valor valido: \n");
        mesesPagoCliente = datosCliente.nextInt();
    }



    if (contadorErrores < 3){System.out.println("\nIngrese la tasa de
        interes");
        numeroValido = datosCliente.hasNextDouble();
    }

    while (numeroValido != true ){
        if (++contadorErrores >= 3){ break;}
        System.out.println("\nEl valor '" + datosCliente.next()  + "'
            Es un caracter, dato no valido, ingrese otro: \n");
        numeroValido = datosCliente.hasNextDouble();
    }

```

```
32      if (contadorErrores < 3 ){tasaDeInteres = datosCliente.nextDouble()
            ;
33      }
34
35      while(tasaDeInteres < 0 && contadorErrores < 3){
36          if (++contadorErrores >= 3){ break;}
37          System.out.println("\nEl valor '" + tasaDeInteres  + "' Es
                negativo, dato no valido, ingrese otro: \n");
38          tasaDeInteres = datosCliente.nextDouble();
39      }
```

El siguiente bloque de código:

```
1           if (estadoCliente == 'P'){
2                       tasaAnual = (tasaDeInteres-2.0)/100.0;
3       }
4
5       else if (estadoCliente == 'E'){
6                       tasaAnual = (tasaDeInteres)/100.0;
7       }
8
9       else if (estadoCliente == 'R'){
10                      tasaAnual = (tasaDeInteres+2.0)/100.0;
11      }
12
13      interesSimple = capitalPrestado*(tasaAnual)*(mesesPagoCliente/12.0)
            ;
14      totalSimple = interesSimple + capitalPrestado;
15      totalCompuesto = capitalPrestado*Math.pow(1+ tasaAnual/12.0,
            mesesPagoCliente);
16      interesCompuesto = totalCompuesto - capitalPrestado;
```

No es más que los cálculos del interés compuesto y simple, que realizamos en la `Practica05`, solo que ahora utilizamos las variables `interesSimple` y `InteresCompuesto` para guardar los respectivos intereses y `totalSimple` y `totalCompuesto` para guardar el total a pagar dependiendo del interés utilizado. La decisión de calcularlo ahora y no después con las opciones del posterior menú, es para no realizar el cálculo en cada opción que lo necesitemos, si llegamos a ocupar el valor, solo basta utilizarlo con la variable que tiene guardado el mismo.

Para ejecutar el menú, primero tenemos el siguiente `while`:

```
1       while (contadorErrores < 3  && operacionEfectiva < 5){
```

Si de lo realizado antes del `while`, obtuvimos que los datos ingresados ya habían sido erróneos un total de 3 veces, entonces ni siquiera dejará ejecutar el menú, el programa terminará mostrando un resumen defecto y dirá en terminal que el programa termino debido al hecho que se superaron la cantidad de errores permitidos. Al igual, si ya se realizaron muchas ejecuciones efectivas, el señor **Pines**, solicito que no se estuviera "jugando" con el programa, por lo que después de pedir 5 veces algo al programa (donde la opción **0)**, que es la de salir del programa no cuenta como una ejecución efectiva realizada) el programa terminará. Si se cumple ambas cosas, tendremos lo siguiente:

```
1       contadorErrores =0;
2   System.out.println("\n\n====SISTEMA DE PRESTAMOS====");
3
```

```
System.out.println("\nOpciones disponiles: ");
System.out.println("\n1) Cotizar con interes simple.");
System.out.println("\n2) Cotizar con interes compuesto.");
System.out.println("\n3) Comparar interes simple y compuesto.");
System.out.println("\n4) Cambiar el capital del prestamo");
System.out.println("\n0) Salir.");
System.out.println("\nPor Favor elige una opcion: ");

numeroValido = datosCliente.hasNextInt();

    while (numeroValido != true ){
        if (++contadorErrores >= 3){ break;}
        System.out.println("\nEl valor '" + datosCliente.next()  + "'
           No es valido, ingrese otro: \n");
        numeroValido = datosCliente.hasNextInt();
    }

    if (contadorErrores < 3 ){opcionMenu = datosCliente.nextInt();
    }

    while(opcionMenu < 0 ){
        if (++contadorErrores >= 3){ break;}
        System.out.println("-Valor negativo, opcion no valida.\n");
        opcionMenu = datosCliente.nextInt();
    }
```

El programa imprime en terminal, que opciones existen y nos pide cual queremos ejecutar para manipular los datos registrados. La variable `contadorErrores` la colocamos en 0 para que en cada ciclo, tenga tres oportunidades de ingresar una opción válida, pues de cualquier manera, el programa acabará después de 5 ejecuciones efectivas y es normal cometer errores al ingresar una opción. Al igual que cuando se ingresa la cantidad de dinero requerida para el prestamos, los `while` del último bloque de código, tienen exactamente la misma lógica, solo complementamos después con :

```
    if (contadorErrores == 3 ){break;}
```

para parar el ciclo `while` principal (es decir el que deja ingresar al menú) por límite de errores. Lo que sigue del código es simplemente lo que hacemos en cada opción:

```
    if (opcionMenu == 1 || opcionMenu == 2){

    if (opcionMenu ==1){
    System.out.println("\n\n====Opcion 1 : Cotizacion de interes simple
       ====\n\n");}

    else if (opcionMenu ==2){
    System.out.println("\n\n====Opcion 2 : Cotizacion de interes
       Compuesto====\n\n");}



    System.out.println("Nombre : "+ nombreCliente);

    if(estadoCliente == 'E'){ System.out.println("Tipo de Cliente:
       Estandar");}
```

```
14      else if(estadoCliente == 'P'){ System.out.println("Tipo de Cliente:
            Preferente");}
15      else if(estadoCliente == 'R'){ System.out.println("Tipo de Cliente:
            Riesgoso");}
16
17      System.out.println("Capital de prestamo: "+ capitalPrestado);
18
19      System.out.println("Tasa anual: "+ tasaDeInteres);
20      System.out.println("Plazo a meses: "+ mesesPagoCliente);
21
22
23
24      if (opcionMenu == 1){
25
26          System.out.printf("\nInteres simple: $%.2f\n", interesSimple);
27          System.out.printf("\nTotal con interes simple: $%.2f\n",
               totalSimple);
28      }
29          else if (opcionMenu ==2){
30      saldoPorMes=capitalPrestado;
31      for (int i=0; i<mesesPagoCliente; i++){
32
33                      saldoPorMes *= (1 +  tasaAnual/12.0);
34                      System.out.printf("En el mes %d se tiene un saldo
                           de: %.2f\n",i+1, saldoPorMes);
35               }
36
37
38          System.out.printf("\nInteres compuesto: $%.2f\n",
               interesCompuesto);
39          System.out.printf("\nTotal con interes compuesto: $%.2f\n",
               totalCompuesto);
40
41          }
42
43  operacionEfectiva++;
44  }
```

Unimos la opción 1 y 2, en un solo `if`, imprimiendo correctamente lo que corresponde, así nos ahorramos escribir la implementación de cada opción, nos podemos tomar está libertad al ser opciones muy parecidas. la impresión de `saldoPorMes`, no es más que la realizada en la `Practica05`. Lo más interesante de este bloque de código, es `operacionEfectiva++`, que lleva el conteo de las operaciones realizadas en el programa. En cada bloque de código `if` de las opciones del menú (a excepción de **0**)), se encuentra un `operacioEfectiva++` para llevar el conteo de las ejecuciones realizadas. Luego:

```
1       else if (opcionMenu == 3){
2       System.out.printf("\nInteres simple: $%.2f\n", interesSimple);
3
4       System.out.printf("\nInteres compuesto: $%.2f\n", interesCompuesto)
           ;
5
6       System.out.printf("\nLa diferencia entre ellos es: $%.2f\n", Math.
           abs(interesCompuesto - interesSimple));
```

```
7
8       if(interesCompuesto > interesSimple){
9           System.out.println("El interes compuesto es mayor");
10      }
11
12      else {
13          System.out.println("El interes simple es mayor");
14      }
15
16          operacionEfectiva++;
17  }
```

Simplemente muestra la comparación del interés simple y el compuesto, con impresiones y operaciones comunes. Seguimos con:

```
1       else if (opcionMenu == 4){
2
3   System.out.println("Ingrese el nuevo prestamo: ");
4
5       while (datosCliente.hasNextDouble() != true ){
6
7           System.out.println("\nEl valor '" + datosCliente.next()  + "'
               No es valido, ingrese otro: \n");
8       }
9
10      capitalPrestado = datosCliente.nextDouble();
11
12
13      while(capitalPrestado < 0 ){
14          System.out.println("-Valor no valido para la cantidad
               necesitada de prestamo.\n");
15          capitalPrestado = datosCliente.nextDouble();
16      }
17
18
19  if (estadoCliente == 'P'){
20                      tasaAnual = (tasaDeInteres-2.0)/100.0;
21      }
22
23      else if (estadoCliente == 'E'){
24                      tasaAnual = (tasaDeInteres)/100.0;
25      }
26
27      else if (estadoCliente == 'R'){
28                      tasaAnual = (tasaDeInteres+2.0)/100.0;
29      }
30
31      interesSimple = capitalPrestado*(tasaAnual)*(mesesPagoCliente/12.0)
           ;
32      totalSimple = interesSimple + capitalPrestado;
33      totalCompuesto = capitalPrestado*Math.pow(1+ tasaAnual/12.0,
           mesesPagoCliente)
34      interesCompuesto = totalCompuesto - capitalPrestado;
35
```

```
36
37
38      System.out.printf("\nInteres simple: $%.2f\n", interesSimple);
39
40      System.out.printf("\nInteres compuesto: $%.2f\n", interesCompuesto)
            ;
41
42      System.out.printf("\nLa diferencia entre ellos es: $%.2f\n", Math.
            abs(interesCompuesto - interesSimple));
43
44
45      System.out.printf("\nTotal con interes simple: $%.2f\n",
            totalSimple);
46      System.out.printf("\nTotal con interes compuesto: $%.2f\n",
            totalCompuesto);
47
48  operacionEfectiva++;
49  }
```

Pide de nuevo el valor de `capitalPrestado`, comprueba que el dato sea válido, y realiza de nuevo toda la operación de calculo vista anteriormente. Finalmente para el menú:

```
1  else if (opcionMenu == 0){
2  break;
3  }
```

Acaba con el `while` principal mediante el `break`. Para terminar todo el código en general, tenemos:

```
1       System.out.println("\n\n====RESUMEN DEL PROGRAMA====\n");
2       System.out.println("Nombre : "+ nombreCliente);
3       System.out.println("Numero de errores en la ultima ejecucion "+
            contadorErrores);
4       System.out.println("Numero de operaciones efectivas en la ultima
            ejecucion "+ operacionEfectiva);
5       System.out.println("Capital de prestamo: "+ capitalPrestado);
6
7
8   if (contadorErrores >= 3){
9       System.out.println("\n--Programa terminado por cantidad de errores
            --");
10  }
11
12  else if (operacionEfectiva == 5) {
13      System.out.println("\n--Programa terminado por cantidad de
            operaciones efectivas--");
14  }
15
16  else {
17  System.Out.println("\n--Programa terminado por el usuario--");
18  }
```

Imprime el resumen del programa y analiza porque razón el mismo dejo de ejecutarse. Para verificar el programa, vamos a realizar las siguientes ejecuciones:

- **Programa con datos de Robbie**: Para el caso de Robbie, ingresamos los siguientes datos:

    1. Nombre: Robbie valentino
    2. Tipo de cliente: E
    3. Capital de préstamo: 12899.00
    4. Tasa Anual: 15
    5. Plazo en meses: 21

    Los datos anteriormente ingresados, efectivamente son correctos al ponerlos en terminal, y usando las distintas opciones (1,2 y 3) nos dan los resultados esperados según la **Resultado1.png** y la **Resultado2.png**. Si vemos las imagenes **ProgramaPines1.png**, **ProgramaPines2.png**, **ProgramaPines3.png** e **ProgramaPines4.png**, observaremos que efectivamente la ejecución fue exitosa.

- **Datos propios**: Para los datos ingresados, según el precio de la laptop soñada en la `Practica02`, tenemos lo siguiente:

    1. Nombre: Guillermo Mejia
    2. Tipo de cliente: P
    3. Capital de préstamo: 66679.00
    4. Tasa Anual: 15
    5. Plazo en meses: 17

    Al utilizar distintas opciones y en especial la opción **4**, obtenemos datos congruentes que se pueden revisar en las Imagenes **ProgramaPines5.png**, **ProgramaPines6.png**, **ProgramaPines7.png**, e **ProgramaPines8.png**.

- **Ingresar datos erróneos**: Primero, ingresamos los siguientes datos:

    1. Nombre: Guillermo Mejia
    2. Tipo de cliente: A

    Lo cual nos dará un error y pedirá un nuevo dato correcto, ingresamos "R" y ahora damos otros datos:

    1. Nombre: Guillermo Mejia
    2. Tipo de cliente: R
    3. Capital de préstamo: 66679.00
    4. Tasa Anual: -15
    5. Plazo en meses: 17

    Lo cual arrojará otro error, pues es un número negativo, para corregirlo ponemos un dato válido, como por ejemplo de nuevo 15 y continuamos hasta la parte del menú, ahí, ingresaremos solo datos erróneos, como caracteres y números negativos hasta que el programa llegue al límite de errores y termine con la ejecución. En las imagenes **ProgramaPines9.png** e **ProgramaPines10.png** verán el resumen del programa después de esta ejecución.

Actividad 2.2 - Conclusión y retrospectiva

Para terminar, en está parte, contestaremos lo solicitado en las instrucciones de:

`P6_Entrada_de_datos_y_cierre_de_la_Cabaña_del_MIsterio.pdf`

1. **Conclusión**: A pesar de que a simple vista, viendo la teoría de clase y revisando la documentación de `Java`, puede resultar sencillo utilizar herramientas como `Scanner`, lo cierto es, que debemos entender que hacer cada uno de los métodos que utilizamos para conseguir datos en terminal, cualquier mal empleo, como el hecho de que `hasNextInt()` analiza valores de verdad a diferencia de `nextInt()` que guarda el entero ingresado, puede crear complicaciones al momento de verificar que los datos que los usuarios colocan sean válidos

   Utilizar los distintos métodos y sentencias de control aprendidos con anterioridad, ayuda a resolver las problemáticas planteadas, con sus distintas dificultadas, como saber, cuando podemos utilizar las distintas sentencias, como llevar un control correcto de las variables (como el hecho de que si utilizamos alguna dentro de un bloque condicional, `Java` puede detectar que no esta inicializada) y como llevar el conteo de nuestras variables de control, para que los ciclos no ejecuten cosas raras o se salgan de control.

2. `Retrospectiva`: Dando una pequeña leída a todas las practicas realizadas, hemos podido notar, como poco a poco, los errores cometidos en un principio, son corregidos mediante la práctica. Si bien la teoría es la que nos nutre del conocimiento necesario para programar, el realizar programas, poner "manos a la obra", no hace más que reforzar el aprendizaje y notar en que podemos equivocarnos y como corregir esos malos hábitos de la programación. Causa bastante felicidad saber, que cada esfuerzo, permite avanzar y no quedarse en el mismo lugar en que empezamos, siempre es importante saber y entender, que no lo sabemos todo.

3. **Programa dentro de main**: Estas semanas han hecho a muchos de nosotros, acostumbrarnos a programar en `main` y manipularlo de la manera más correcta posible, cuando un programa es corto, el problema de identificar errores se ha vuelto cotidiano y fácil de notar, pero, en está última `Practica06`, hubo demasiados momentos en lo que intentar corregir una parte de código, llevaba un tiempo y concentración un poco innecesario, buscar donde borrar, que escribir o que llave poner en que lugar, era una tarea bastante ardua, si te pierdes en algún momento entre tanto texto. Además, muchas de las ejecuciones `while` y cálculos requeridos, eran partes de código bastante repetidas, dándonos a pensar, la manera en que podríamos ahorrarnos esta escritura, simplemente programandolo una ves, y utilizando cada que lo necesitemos.